

Ai Recommendations

The RM looks at a client's portfolio and asks: *"Based on what's happening in the market right now, what should I do with each sector this client holds?"*

Output per sector:

Semiconductors → BUY more "AI chip demand surge supports further accumulation"

Financials → HOLD "Rate hold creates stability, no action needed"

Renewable Energy → SELL "ECB rate pressure reducing green bond valuations"

Where the data already lives:

Client holdings → which sectors the client is in (from portfolios → holdings → security → sector)

ClientThemeMatch → which themes match those sectors + sentiment (Positive/Negative/Neutral)

ClientOutlook → drivers with status: Increase / Decrease / Neutral ← KEY

SectorTag → sector-level sentiment from AI processing

Architecture:

INPUTS (all from existing DB)

1. Client holdings → extract unique sectors client is exposed to
2. ClientOutlook.drivers → [{title, commentary, status: Increase/Decrease/Neutral}]
3. ClientThemeMatch → themes matched to client + sentiment per theme
4. SectorTag → sector-level sentiment from step 2 processing
5. ClientPersonalDetails.client_constraints → hard rules

LOGIC (pure Python — no LLM needed)

For each sector the client holds:

Step 1: Collect all signals for that sector

- Outlook driver status (Increase/Decrease/Neutral) → weight: 0.50(0.2)
- Theme sentiment for themes touching this sector → weight: 0.30
- SectorTag sentiment from processed news → weight: 0.20(make higher 0.5)

Step 2: Compute signal score

Increase/Positive = +1

Neutral = 0

Decrease/Negative = -1

Step 3: Weighted score → action

score > 0.3 → BUY

score < -0.3 → SELL

otherwise → HOLD

Step 4: Generate 1-line description

Use LLM (or template) referencing the actual driver/theme that drove the decision

OUTPUT (new DB table: ClientSectorRecommendation)

client_id

sector_name → "Semiconductors"

action → "BUY" / "HOLD" / "SELL"

signal_score → 0.75 (for transparency)
description → "AI chip demand surge and ASML position support
accumulation"
generated_at

New table needed:

ClientSectorRecommendation — one row per client per sector.

New file: step8_recommend.py (rewritten) — pure sector-signal logic + one LLM call per client for descriptions.

api.py — one new endpoint: GET /clients/{id}/sector-recommendations

models.py — replacing ClientFundRecommendation with ClientSectorRecommendation

Flow in pipeline:

Step 3 (match) → Step 5 (outlook) → Step 6 (sector recommendations)

Step 6 reads step 3 and step 5 output → produces sector-level BUY/HOLD/SELL → saves to DB.

Need to do:

sectors

- score - weighted score

sector 1 - 0.7

sector 2 - 0.2

sector 3 - 0.01

sector 4 - -0.4

threshold - 0.3

sector - funds in that sector

sector - 2 sectors qualified for buy (>0.3)

- 1 sector qualified for sell (<0.1 or negative)

Buy

sector 1 - 0.7 - 8 funds

sector 2 - 0.56 - 10 funds

prompt

<client,...>

<funds>

Part 1 — Already built (Step 6)

What to do with sectors the client **already holds**:

Semiconductors → BUY more "AI chip demand surge supports accumulation"

Financials → HOLD "Rate hold creates stability"

Renewable Energy → SELL "ECB pressure reducing green bond valuations"

This is done. 

Part 2 — New: What new funds/securities to invest in

The question here is different:

"The client doesn't hold X sector yet — but based on market signals, should they?"

"Within a sector they hold, are there specific securities better than what they have?"

Analysis: Where does this data come from?

What we have in the DB:

- SectorTag → sectors with Positive sentiment from current news
- ClientThemeMatch → themes with Increase sentiment the client is exposed to
- ClientOutlook.drivers → Increase-status drivers pointing to sectors
- ClientPreference → allowed security type bandwidths (EQUITY/FIXED_INCOME etc.)
- ClientPersonalDetails.client_constraints → hard rules (ESG only, no crypto etc.)
- Security table → all known securities with their sectors
- SecurityPriceHistory → price trend data per ticker

What we DON'T have:

- A fund universe (no external fund DB — we only have 10 securities in seed_clients.py)
 - Real-time fund performance data
 - Fund expense ratios, categories, NAV
-

Two design options:

Option A — Recommend from existing Security table (what you have now)

Logic:

1. Find sectors with strong Increase signal (from SectorTag + drivers)
2. Find sectors client is NOT currently exposed to (or underweight)
3. Look up Securities in those sectors from Security table
4. Rank by price momentum (SecurityPriceHistory trend)
5. Recommend top 2-3 securities the client should consider adding

Output:

NEW INVESTMENT OPPORTUNITIES

NVDA (Semiconductors) "Strong AI tailwind — not in your portfolio yet"

ALT2 (Renewable Energy) "EU taxonomy expansion — underweight vs preference"

Pros: uses only existing data, no external API needed.

Cons: only 10 securities in DB — limited universe.

Option B — Static fund universe in code (like the old step6)

Logic:

1. Same signal scoring as Part 1 but for sectors client is NOT holding
2. Match against a hardcoded fund list (F001-F010 from previous step6)
3. Score and rank funds by: sector signal + client preference fit + risk match
4. Recommend top 3 new funds

Output:

RECOMMENDED NEW FUNDS

ESG Small Cap Innovation Fund "EU taxonomy tailwind aligns with ESG constraint"

Short Duration Green Bond Fund "Rate hold environment favours short duration"

Pros: richer fund universe, can add descriptions, categories, expense ratios.

Cons: hardcoded — not real market data.

Option C — Hybrid (recommended)

Part 1 (Step 6) → existing sectors → BUY/HOLD/SELL

Part 2 (Step 8) → new opportunities → two sub-outputs:

- a) Securities from Security table not currently held → "Gap Analysis"
- b) Fund universe (static) → "New Fund Suggestions"

This gives the RM a complete picture:

- What to do with what they have (Step 6)
 - What new things to add (Step 8)
-

My recommendation: Option C

Step 8 does two things:

8a — GAP ANALYSIS (from Security table)

Sectors with Increase signal + client not holding → suggest securities

8b — FUND SUGGESTIONS (from static fund list)

Match Increase-signal sectors to fund universe → suggest top 3 funds

Respecting: risk profile, preference bandwidths, constraints

Both use the same signal scores already computed in Step 6 — no duplicate work.

New DB table needed: ClientNewInvestmentSuggestion

client_id

suggestion_type → "SECURITY" or "FUND"

ticker_or_fund_id → e.g. "NVDA" or "F005"

name → display name

sector_name → which sector this covers

signal_score → inherited from sector signal

rationale → 1-line LLM description

generated_at

New file: step8_recommend.py

New API endpoint: GET /clients/{id}/new-investments